

Tab 7

TITLE : SYNC SUITES HOTEL
RESERVATION SYSTEM

1- EINS

GROUP NAME : DEVSKADA

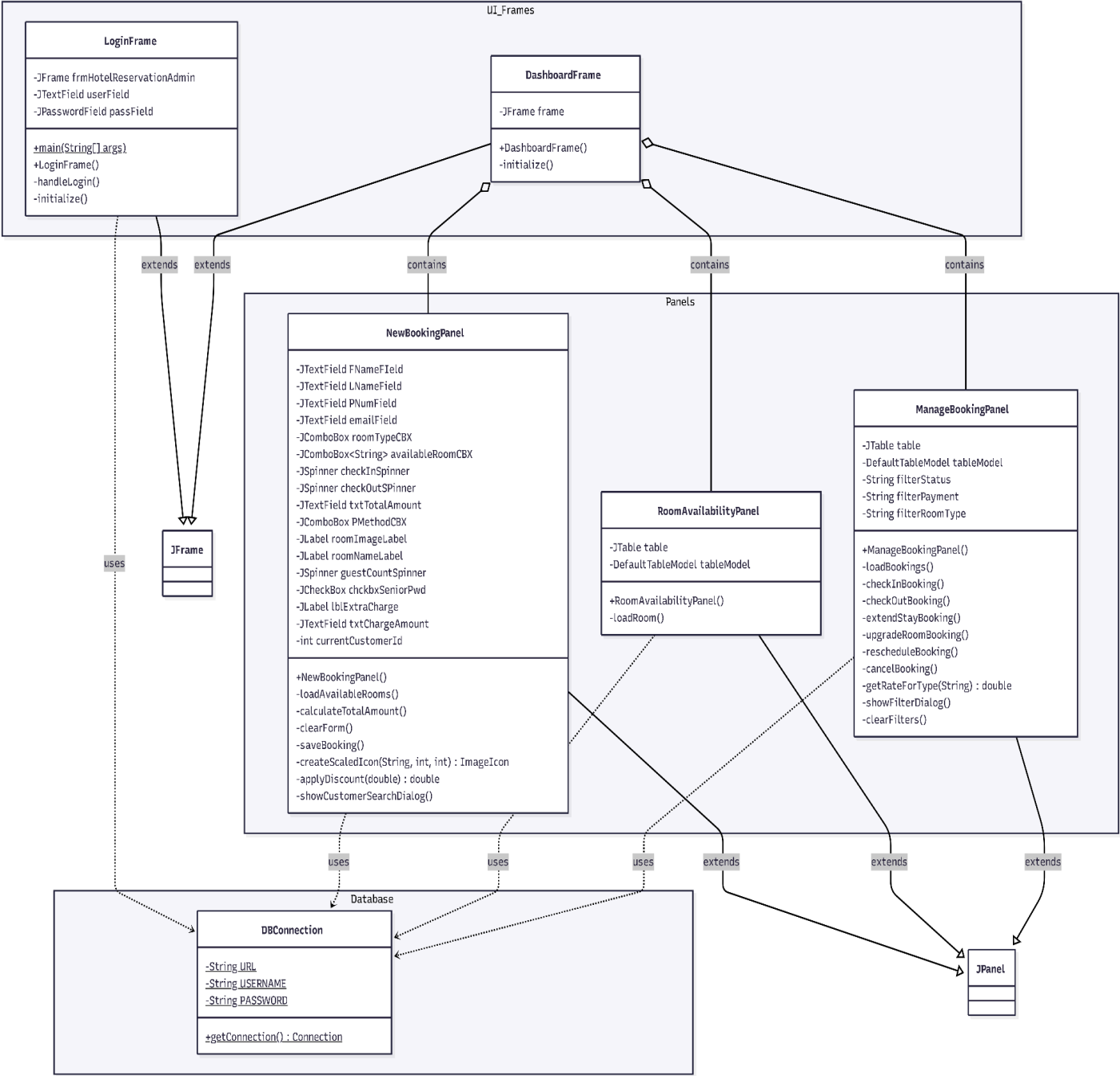
YABUT, RENYER N.

ARJAY LEXIEL ESPIJON

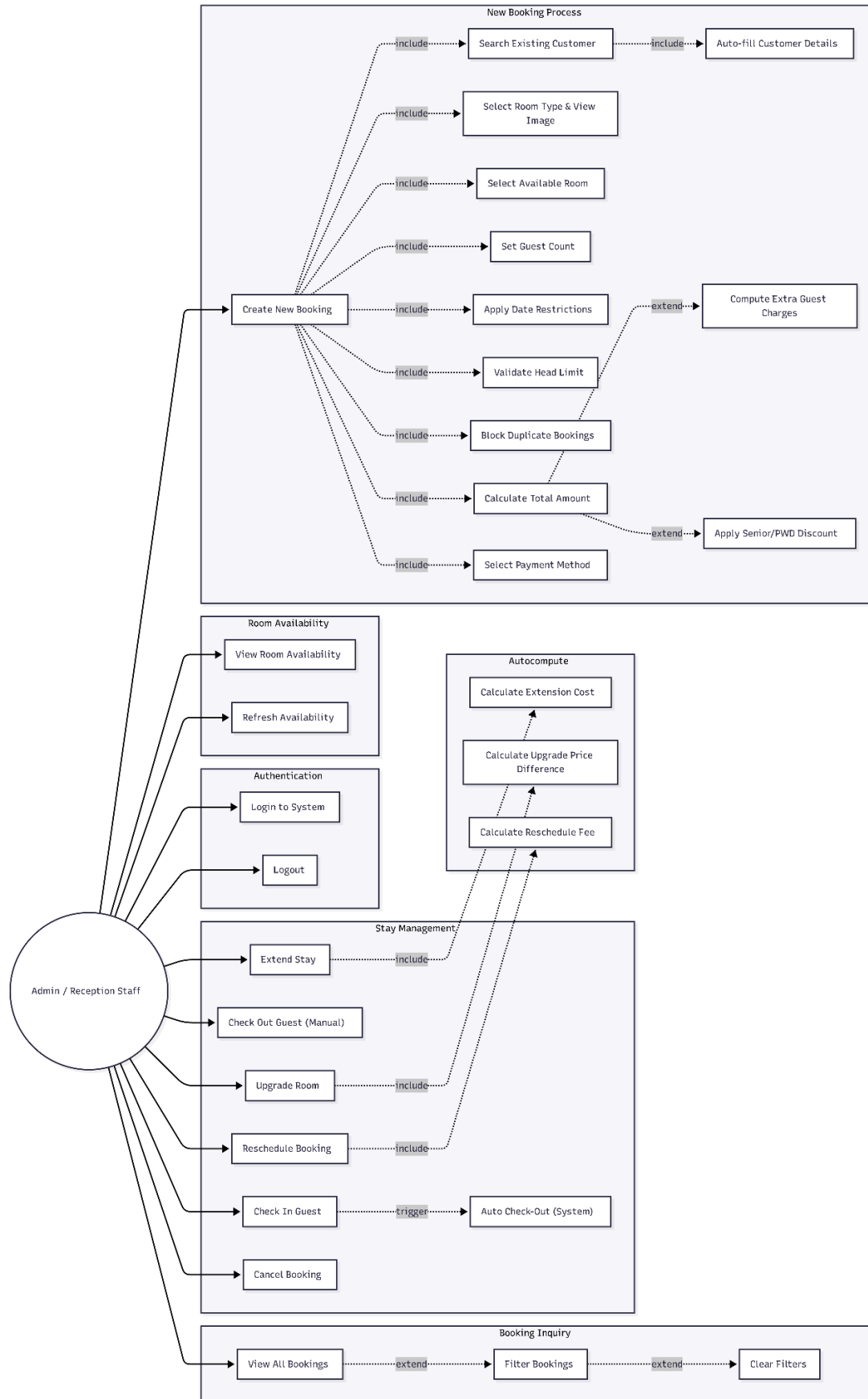
JOSHUA MIGUEL GERTOS

UTRERA, SEAN JOHN DANIEL M.

UML CLASS DIAGRAM



UML CASE DIAGRAM



TABLES

```
MariaDB [hotel_system]> desc admins;
```

Field	Type	Null	Key	Default	Extra
adminID	int(11)	NO	PRI	NULL	auto_increment
username	varchar(50)	NO		NULL	
password	varchar(50)	NO		NULL	

```
3 rows in set (0.042 sec)
```

```
MariaDB [hotel_system]> desc bookings;
```

Field	Type	Null	Key	Default	Extra
booking_id	int(11)	NO	PRI	NULL	auto_increment
customer_id	int(11)	NO	MUL	NULL	
room_id	int(11)	NO	MUL	NULL	
check_in_date	date	NO		NULL	
check_out_date	date	NO		NULL	
total_amount	decimal(10,2)	YES		NULL	
reschedule_charge	decimal(10,2)	YES		0.00	
payment_method	enum('Cash', 'GCash/QR Scan')	NO		Cash	
status	enum('Reserved', 'Checked In', 'Checked Out', 'Cancelled')	NO		Reserved	

```
9 rows in set (0.040 sec)
```

```
MariaDB [hotel_system]> desc customers;
```

Field	Type	Null	Key	Default	Extra
customer_id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
phone_number	varchar(30)	NO		NULL	
email	varchar(50)	NO		NULL	
created_at	datetime	YES		NULL	

```
6 rows in set (0.040 sec)
```

```
MariaDB [hotel_system]> desc room_types;
```

Field	Type	Null	Key	Default	Extra
room_type_id	int(11)	NO	PRI	NULL	auto_increment
type_name	varchar(50)	NO		NULL	
rate_per_day	decimal(10,2)	NO		NULL	

```
3 rows in set (0.074 sec)
```

```
MariaDB [hotel_system]> desc rooms;
```

Field	Type	Null	Key	Default	Extra
room_id	int(11)	NO	PRI	NULL	auto_increment
room_number	varchar(10)	NO	UNI	NULL	
room_type_id	int(11)	NO	MUL	NULL	
is_available	tinyint(1)	YES		1	

```
4 rows in set (0.017 sec)
```

```
MariaDB [hotel_system]> |
```

Database Schema

ADMINS		
int	adminID	PK
varchar	username	
varchar	password	

ROOM_TYPES		
int	room_type_id	PK
varchar	type_name	
decimal	rate_per_day	

ROOMS		
int	room_id	PK
varchar	room_number	
int	room_type_id	FK
tinyint	is_available	

CUSTOMERS		
int	customer_id	PK
varchar	first_name	
varchar	last_name	
varchar	phone_number	
varchar	email	
datetime	created_at	

BOOKINGS		
int	booking_id	PK
int	customer_id	FK
int	room_id	FK
date	check_in_date	
date	check_out_date	
decimal	total_amount	
decimal	reschedule_charge	
enum	payment_method	
enum	status	

categorizes

assigned to

places

Program source code

```
Command Prompt - mysql -u X + v
MariaDB [(none)]> create database hotel_system;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> use hotel_system;
Database changed
MariaDB [hotel_system]> CREATE TABLE admins (
->     adminID INT(11) NOT NULL AUTO_INCREMENT,
->     username VARCHAR(50),
->     password VARCHAR(60),
->     PRIMARY KEY (adminID)
-> );
Query OK, 0 rows affected (0.009 sec)

MariaDB [hotel_system]> CREATE TABLE bookings (
->     booking_id INT(11) NOT NULL AUTO_INCREMENT,
->     customer_id INT(11) NOT NULL,
->     room_id INT(11) NOT NULL,
->     check_in_date DATE NOT NULL,
->     check_out_date DATE NOT NULL,
->     total_amount DECIMAL(10,2),
->     reschedule_charge DECIMAL(10,2) DEFAULT 0.00,
->     payment_method ENUM('Cash','GCash/QR Scan'),
->     status ENUM('Reserved','Checked In','Checked Out','Cancelled')
->     DEFAULT 'Reserved',
->     PRIMARY KEY (booking_id)
-> );
Query OK, 0 rows affected (0.036 sec)
```

```
Command Prompt - mysql -u X + v
MariaDB [hotel_system]> CREATE TABLE customers (
->     customer_id INT(11) NOT NULL AUTO_INCREMENT,
->     first_name VARCHAR(50) NOT NULL,
->     last_name VARCHAR(50) NOT NULL,
->     phone_number VARCHAR(30) NOT NULL,
->     email VARCHAR(50) NOT NULL,
->     created_at DATETIME,
->     PRIMARY KEY (customer_id)
-> );
Query OK, 0 rows affected (0.024 sec)

MariaDB [hotel_system]> CREATE TABLE room_types (
->     room_type_id INT(11) NOT NULL AUTO_INCREMENT,
->     type_name VARCHAR(50) NOT NULL,
->     rate_per_day DECIMAL(10,2) NOT NULL,
->     PRIMARY KEY (room_type_id)
-> );
Query OK, 0 rows affected (0.011 sec)

MariaDB [hotel_system]> CREATE TABLE rooms (
->     room_id INT(11) NOT NULL AUTO_INCREMENT,
->     room_number VARCHAR(10) NOT NULL UNIQUE,
->     room_type_id INT(11) NOT NULL,
->     available TINYINT(1) DEFAULT 1,
->     PRIMARY KEY (room_id),
->     FOREIGN KEY (room_type_id) REFERENCES room_types(room_type_id)
-> );
Query OK, 0 rows affected (0.031 sec)
```


1. DB CONNECTION

```
1 package Hotel_Reservation;
2 import java.sql.Connection;
3
4
5
6 public class DBConnection {
7
8     // Pointing to out database
9     private static final String URL = "jdbc:mysql://localhost:3306/hotel_system"; // change "hotel_reservation"
10    private static final String USERNAME = "root";
11    private static final String PASSWORD = ""; // leave empty unless you change your password
12
13    public static Connection getConnection() throws SQLException {
14        try {
15            Class.forName("com.mysql.cj.jdbc.Driver");
16            return DriverManager.getConnection(URL, USERNAME, PASSWORD);
17        } catch (ClassNotFoundException e) {
18            throw new SQLException("MySQL JDBC Driver not found. Check your build path!", e);
19        }
20    }
21
22
23 }
24
```

2. DASH BOARD FRAME

```
1 package Hotel_Reservation;
2 import javax.swing.*;
3 import java.awt.*;
4 import java.sql.Connection;
5 import java.sql.PreparedStatement;
6 import java.sql.ResultSet;
7 import java.sql.SQLException;
8 import java.awt.event.ActionListener;
9 import java.awt.event.ActionEvent;
10
11 public class DashboardFrame extends JFrame {
12
13     private JFrame frame;
14
15
16     /**
17      * Create the application.
18      */
19     public DashboardFrame() {
20         initialize();
21     }
22
23     /**
24      * Initialize the contents of the frame.
25      */
26     private void initialize() {
27         frame = new JFrame();
28         frame.setUndecorated(true);
29         frame.setExtendedState(JFrame.MAXIMIZED_BOTH);
30         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31         frame.getContentPane().setLayout(new BorderLayout(0, 0));
32
33         JPanel panel = new JPanel();
34         panel.setBackground(new Color(39, 0, 110));
35         panel.setPreferredSize(new Dimension(900, 600));
36         frame.getContentPane().add(panel, BorderLayout.NORTH);
37         panel.setLayout(new BorderLayout(0, 0));
38
39         JLabel lblHotelReservationSystem = new JLabel("Sync Suites Hotel ");
40         lblHotelReservationSystem.setFont(new Font("SansSerif", Font.BOLD, 20));
41     }
42 }
```

```

41     lblHotelReservationSystem.setHorizontalAlignment(SwingConstants.RIGHT);
42     lblHotelReservationSystem.setForeground(new Color(255, 255, 255));
43     panel.add(lblHotelReservationSystem, BorderLayout.WEST);
44
45     JPanel panel_1 = new JPanel();
46     panel_1.setBackground(new Color(39, 0, 110));
47     panel_1.setForeground(new Color(255, 255, 255));
48     panel.add(panel_1, BorderLayout.EAST);
49
50     JButton LogoutBtn = new JButton("Logout");
51     LogoutBtn.addActionListener(new ActionListener() {
52         public void actionPerformed(ActionEvent e) {
53             int confirm = JOptionPane.showConfirmDialog(frame, "Are you sure you want to logout?", "Confirm Logout", JOptionPane.YES_NO_OPTION);
54             if (confirm == JOptionPane.YES_OPTION) {
55                 frame.dispose();
56                 new LoginFrame();
57             }
58         }
59     });
60     LogoutBtn.setForeground(new Color(255, 255, 255));
61     LogoutBtn.setFont(new Font("SansSerif", Font.BOLD, 14));
62     LogoutBtn.setBackground(new Color(224, 27, 36));
63     panel_1.add(LogoutBtn);
64
65     JTabbedPane tabbedPane = new JTabbedPane(JTabbedPane.TOP);
66     frame.getContentPane().add(tabbedPane, BorderLayout.CENTER);
67
68     NewBookingPanel panel_2 = new NewBookingPanel();
69     panel_2.setName("");
70     tabbedPane.addTab("New Booking", null, panel_2, null);
71
72     RoomAvailabilityPanel panel_3 = new RoomAvailabilityPanel();
73     tabbedPane.addTab("Room Availability", null, panel_3, null);
74
75     JPanel panel_4 = new ManageBookingPanel();
76     tabbedPane.addTab("Manage Booking", null, panel_4, null);
77
78     frame.setVisible(true);
79 }
80

```

3. ROOM AVAILABILITY OVERVIEW

```

10 import javax.swing.*;
11
12 public class RoomAvailabilityPanel extends JPanel {
13     private static final long serialVersionUID = 1L;
14     private JTable table;
15     private DefaultTableModel tableModel;
16
17     /**
18      * Create the panel.
19      */
20     public RoomAvailabilityPanel() {
21         setLayout(new BorderLayout());
22         //title
23         JLabel titleLabel = new JLabel("Room Availability Overview");
24         titleLabel.setForeground(new Color(0, 0, 60));
25         titleLabel.setFont(new Font("SansSerif", Font.BOLD, 20));
26         titleLabel.setBorder(new EmptyBorder(15, 15, 10, 0));
27         add(titleLabel, BorderLayout.NORTH);
28
29         //table
30         //column name
31         String[] columns = {"Room No.", "Room Type", "Rate / Day (P)", "Status"};
32         tableModel = new DefaultTableModel(columns, 0) {
33             @Override
34             public boolean isCellEditable(int row, int col) {
35                 return false; // so the tables is not editable and it will be read only
36             }
37         };
38
39         JScrollPane scrollPane = new JScrollPane();
40         add(scrollPane, BorderLayout.CENTER);
41
42         table = new JTable(tableModel);
43     }
44 }

```

```

42     table = new JTable(tableModel);
43     scrollPane.setViewportView(table);
44     table.setRowHeight(28);
45     table.getTableHeader().setFont(new Font("SanSerif", Font.BOLD, 13)); //table header font
46     table.getTableHeader().setForeground(new Color(255, 255, 255)); //table header text color
47     table.getTableHeader().setBackground(new Color(31, 71, 145)); //table header background color
48     table.setFont(new Font("SansSerif", Font.PLAIN, 13));
49
50
51     //south panel or the submit button
52     JPanel panel = new JPanel();
53     add(panel, BorderLayout.SOUTH);
54
55     JButton refreshBtn = new JButton("Refresh");
56     refreshBtn.addActionListener(new ActionListener() {
57         public void actionPerformed(ActionEvent e) {
58             loadRoom();
59         }
60     });
61     refreshBtn.setFont(new Font("SansSerif", Font.BOLD, 12));
62     refreshBtn.setForeground(new Color(0, 0, 0));
63     refreshBtn.setBackground(UIManager.getColor("Button.highlight"));
64     panel.setLayout(new FlowLayout(FlowLayout.RIGHT));
65
66
67     panel.add(refreshBtn);
68
69     loadRoom();
70 }
71 private void loadRoom() {
72     tableModel.setRowCount(0);
73     try (Connection conn = DBConnection.getConnection()){
74         String sql = "SELECT r.room_number, rt.type_name, rt.rate_per_day, " +
75             "CASE " +
76             "    WHEN b.status = 'Checked In' THEN 'Occupied' " +
77             "    WHEN b.status = 'Reserved' THEN 'Reserved' " +
78             "    ELSE 'Available' " +
79             "END AS status " +
80             "FROM rooms r " +
81             "JOIN room_types rt ON r.room_type_id = rt.room_type_id " +

```

```

82             "JOIN room_types rt ON r.room_type_id = rt.room_type_id " +
83             "LEFT JOIN bookings b ON r.room_id = b.room_id " +
84             "AND b.status IN ('Reserved', 'Checked In') " +
85             "ORDER BY " +
86             "CASE " +
87             "    WHEN b.status = 'Checked In' THEN 1 " +
88             "    WHEN b.status = 'Reserved' THEN 2 " +
89             "    ELSE 3 " +
90             "END DESC, " + // DESC para Occupied muna, then Reserved, then Available
91             "r.room_type_id, r.room_number";
92
93     Statement stmt = conn.createStatement();
94     ResultSet rs = stmt.executeQuery(sql);
95
96     while (rs.next()) {
97         tableModel.addRow(new Object[] {
98             rs.getString("room_number"),
99             rs.getString("type_name"),
100             String.format("%.2f", rs.getDouble("rate_per_day")),
101             rs.getString("status")
102         });
103     }
104 } catch (SQLException e) {
105     JOptionPane.showMessageDialog(this, "Error loading rooms: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
106 }
107 }
108
109 }

```

4. NEW BOOKING PANEL

```
10 import java.awt.*;
11
12 public class NewBookingPanel extends JPanel {
13
14     private static final long serialVersionUID = 1L;
15     private JTextField fNameField;
16     private JTextField lNameField;
17     private JTextField pNumField;
18     private JTextField emailField;
19     private JComboBox roomTypeCBX;
20     private JComboBox availableRoomCBX;
21     private JSpinner checkInSpinner;
22     private JSpinner checkOutSpinner;
23     private JTextField txtTotalAmount;
24     private JComboBox pMethodCBX;
25     private JLabel roomImageLabel;
26     private JLabel roomNameLabel;
27     // ADDED: para sa number of guests
28     private JSpinner guestCountSpinner;
29     private JCheckBox chkcbxSeniorPwd;
30     private JLabel lblExtraCharge;
31     private JTextField txtChargeAmount;
32     private int currentCustomerId = -1;
33
34
35     public NewBookingPanel() {
36         setForeground(new Color(31, 26, 85));
37         setSize(1924, 1083);
38         setLayout(null);
39
40         JLabel lblNewBooking = new JLabel("CREATE NEW BOOKING\n");
41         lblNewBooking.setForeground(new Color(0, 0, 68));
42         lblNewBooking.setBounds(44, 20, 364, 28);
43         lblNewBooking.setFont(new Font("SansSerif", Font.BOLD, 20));
44         add(lblNewBooking);
45
46         //ADDED: search existing customer
47         JButton btnSearchCustomer = new JButton("Search Customer");
48         btnSearchCustomer.setBounds(331, 60, 241, 25);
49         btnSearchCustomer.setFont(new Font("SansSerif", Font.BOLD, 12));
50         btnSearchCustomer.setBackground(new Color(0, 128, 255));
51         btnSearchCustomer.setForeground(Color.WHITE);
52         btnSearchCustomer.addActionListener(e -> showCustomerSearchDialog());
53         add(btnSearchCustomer);
54
55         JLabel lblcostumerInformation = new JLabel("--Customer Information");
56         lblcostumerInformation.setForeground(new Color(0, 0, 68));
57         lblcostumerInformation.setFont(new Font("SansSerif", Font.BOLD, 18));
58         lblcostumerInformation.setBounds(100, 60, 241, 17);
59         add(lblcostumerInformation);
60
61         JLabel lblFirst = new JLabel("First Name:");
62         lblFirst.setFont(new Font("SansSerif", Font.BOLD, 14));
63         lblFirst.setBounds(193, 90, 104, 17);
64         add(lblFirst);
65
66         fNameField = new JTextField();
67         fNameField.setBounds(331, 90, 241, 21);
68         add(fNameField);
69         fNameField.setColumns(10);
70         // ADDED: auto uppercase at block numbers sa First Name
71         fNameField.addKeyListener(new java.awt.event.KeyAdapter() {
72             public void keyTyped(java.awt.event.KeyEvent e) {
73                 char c = e.getKeyChar();
74                 // block numbers - hindi makakapag type ng number
75
76                 if (c >= '0' && c <= '9') {
77                     e.consume();
78                 }
79                 // auto uppercase - kahit hindi pinindot caps lock
80                 if (Character.isLetter(c)) {
81                     e.setKeyChar(Character.toUpperCase(c));
82                 }
83             }
84         });
85
86         JLabel lblLastName = new JLabel("Last Name:");
87         lblLastName.setFont(new Font("SansSerif", Font.BOLD, 14));
88         lblLastName.setBounds(193, 120, 104, 17);
89         add(lblLastName);
90
91         JLabel lblPhoneNumber = new JLabel("Phone Number:");
92         lblPhoneNumber.setFont(new Font("SansSerif", Font.BOLD, 14));
93         lblPhoneNumber.setBounds(193, 150, 128, 17);
94         add(lblPhoneNumber);
95
96         JLabel lblEmail = new JLabel("Email:");
97         lblEmail.setFont(new Font("SansSerif", Font.BOLD, 14));
98         lblEmail.setBounds(200, 180, 121, 17);
99         add(lblEmail);
100
101         JLabel lblroomSelection = new JLabel("--Room Selection");
102         lblroomSelection.setForeground(new Color(0, 0, 68));
103         lblroomSelection.setFont(new Font("SansSerif", Font.BOLD, 18));
104         lblroomSelection.setBounds(100, 210, 214, 17);
105         add(lblroomSelection);
106
107         JLabel lblRoomType = new JLabel("Room type:");
108         lblRoomType.setFont(new Font("SansSerif", Font.BOLD, 14));
109         lblRoomType.setBounds(193, 240, 121, 17);
110     }
111 }
```

```

108 lblRoomType.setBounds(193, 240, 121, 17);
109 add(lblRoomType);
110
111 JLabel lblAvailableRoom = new JLabel("Available Room");
112 lblAvailableRoom.setFont(new Font("SansSerif", Font.BOLD, 14));
113 lblAvailableRoom.setBounds(193, 270, 121, 17);
114 add(lblAvailableRoom);
115
116 // ADDED: No. of Guests label - katabi ng room selection
117 JLabel lblGuestCount = new JLabel("No. of Guests:");
118 lblGuestCount.setFont(new Font("SansSerif", Font.BOLD, 14));
119 lblGuestCount.setBounds(193, 300, 121, 17);
120 add(lblGuestCount);
121
122 JLabel lblDates = new JLabel("---Dates");
123 lblDates.setForeground(new Color(0, 0, 68));
124 lblDates.setFont(new Font("SansSerif", Font.BOLD, 18));
125 lblDates.setBounds(100, 330, 214, 17);
126 add(lblDates);
127
128 JLabel lblCheck = new JLabel("Check-in Date");
129 lblCheck.setFont(new Font("SansSerif", Font.BOLD, 14));
130 lblCheck.setBounds(193, 360, 121, 17);
131 add(lblCheck);
132
133 JLabel lblCheckoutDate = new JLabel("Check-out Date");
134 lblCheckoutDate.setFont(new Font("SansSerif", Font.BOLD, 14));
135 lblCheckoutDate.setBounds(193, 390, 128, 17);
136 add(lblCheckoutDate);
137
138 JLabel lblNewLabel = new JLabel("Payment");
139 lblNewLabel.setForeground(new Color(0, 0, 68));
140 lblNewLabel.setFont(new Font("SansSerif", Font.BOLD, 18));
141

```

```

141 lblNewLabel.setBounds(100, 410, 214, 17);
142 add(lblNewLabel);
143
144 JLabel lblPaymentMethod = new JLabel("Payment Method");
145 lblPaymentMethod.setFont(new Font("SansSerif", Font.BOLD, 14));
146 lblPaymentMethod.setBounds(193, 470, 147, 17);
147 add(lblPaymentMethod);
148
149 JLabel lblCheckOut = new JLabel("Total Amount");
150 lblCheckOut.setFont(new Font("SansSerif", Font.BOLD, 14));
151 lblCheckOut.setBounds(193, 500, 121, 17);
152 add(lblCheckOut);
153
154 LNameField = new JTextField();
155 LNameField.setBounds(331, 120, 241, 21);
156 add(LNameField);
157 LNameField.setColumns(10);
158 // ADDED: auto uppercase at block numbers sa Last Name
159 LNameField.addKeyListener(new java.awt.event.KeyAdapter() {
160     @Override
161     public void keyTyped(KeyEvent e) {
162         char c = e.getKeyChar();
163         // block numbers - hindi makakapag type ng number
164         if (c >= '0' && c <= '9') {
165             e.consume();
166         }
167         // auto uppercase - kahit hindi pinindot caps lock
168         if (Character.isLetter(c)) {
169             e.setKeyChar(Character.toUpperCase(c));
170         }
171     }
172 });
173
174 PNumField = new JTextField();

```

```

174 PNumField = new JTextField();
175 PNumField.addKeyListener(new KeyAdapter() {
176     @Override
177     public void keyTyped(KeyEvent e) {
178         char c = e.getKeyChar();
179
180         //will ignore if not number
181         if (!Character.isDigit(c)) {
182             e.consume();
183         }
184         //para hindi pwedeng lumagpas ng 11 digit
185         if (PNumField.getText().length() >= 11) {
186             e.consume();
187         }
188     }
189 });
190 PNumField.setBounds(331, 150, 241, 21);
191 add(PNumField);
192 PNumField.setColumns(10);
193 //same sa nameField but reverse para naman numbers lang ang pwedeng i type
194
195 emailField = new JTextField();
196 emailField.setBounds(331, 180, 241, 21);
197 add(emailField);
198 emailField.setColumns(10);
199
200 //room type combo box drop down
201 String[] roomTypes = {
202     "Standard Room - ₱2,500/day",
203     "Junior suite - ₱3,500/day",
204     "Executive Suite - ₱5,000/day",
205     "Presidential Suite - ₱8,000/day"
206 }

```

```

287     });
288
289     roomTypeCBX = new JComboBox(roomTypes);
290     roomTypeCBX.addActionListener(new ActionListener() {
291         public void actionPerformed(ActionEvent e) {
292             loadAvailableRooms(); // everytime you change the room types the available room number change also
293             calculateTotalAmount(); // this method will calculate naman yung total amount everytime
294
295             // dto lalabas ung mga pic ng room pag sinelect mo ano gusto mo
296             String selectedRoom = (String) roomTypeCBX.getSelectedItem();
297             String imageFile = "";
298
299             if (selectedRoom.contains("Standard Room")) {
300                 imageFile = "Standard room.jpg";
301                 roomNameLabel.setText("Standard Room");
302             } else if (selectedRoom.contains("Junior")) {
303                 roomNameLabel.setText("Junior Suite");
304                 imageFile = "Junior room.jpg";
305             } else if (selectedRoom.contains("Executive")) {
306                 imageFile = "Executive room.jpg";
307                 roomNameLabel.setText("Executive Suite");
308             } else if (selectedRoom.contains("Presidential")) {
309                 imageFile = "Presidential room.jpg";
310                 roomNameLabel.setText("Presidential Suite");
311             }
312
313             roomImageLabel.setIcon(createScaledIcon(imageFile, 815, 649));
314         }
315     });
316     roomTypeCBX.setBounds(331, 240, 241, 26);
317     add(roomTypeCBX);
318
319     // ADDED: room name label sa taas ng image
320     roomNameLabel = new JLabel("");

```

```

240     roomNameLabel = new JLabel("");
241     roomNameLabel.setBounds(654, 5, 815, 20);
242     roomNameLabel.setFont(new Font("SansSerif", Font.BOLD, 20));
243     roomNameLabel.setForeground(new Color(0, 0, 68));
244     roomNameLabel.setHorizontalAlignment(SwingConstants.LEFT);
245     add(roomNameLabel);
246
247     // ADDED: setup ng image label na lalabas sa tabi ng combobox
248     roomImageLabel = new JLabel();
249     roomImageLabel.setFont(new Font("SansSerif", Font.BOLD, 16));
250     roomImageLabel.setBounds(654, 39, 426, 482);
251     roomImageLabel.setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY));
252     roomImageLabel.setHorizontalAlignment(JLabel.RIGHT);
253     add(roomImageLabel);
254
255     availableRoomCBX = new JComboBox();
256     availableRoomCBX.setBounds(331, 270, 241, 26);
257     add(availableRoomCBX);
258
259     // CHANGED: Replaced hard limits with a standard 1-20 range
260     guestCountSpinner = new JSpinner(new SpinnerNumberModel(1, 1, 20, 1));
261     guestCountSpinner.setBounds(334, 300, 74, 22);
262     // will trigger the calculateTotalAmount method when clicking arrow
263     guestCountSpinner.addChangeListener(e -> calculateTotalAmount());
264     add(guestCountSpinner);
265
266     // display the total charged amount
267     lblExtraCharge = new JLabel("");
268     lblExtraCharge.setFont(new Font("SansSerif", Font.ITALIC, 12));
269     lblExtraCharge.setForeground(new Color(224, 27, 36)); // Red text
270     lblExtraCharge.setBounds(416, 401, 258, 22);
271     add(lblExtraCharge);
272
273     // 1. Get today's date and reset the time to midnight
274     java.util.Calendar today = java.util.Calendar.getInstance();
275     today.set(java.util.Calendar.HOUR_OF_DAY, 0);
276     today.set(java.util.Calendar.MINUTE, 0);
277     today.set(java.util.Calendar.SECOND, 0);
278     today.set(java.util.Calendar.MILLISECOND, 0);
279     Date minDate = today.getTime();
280
281     // 2. Pass minDate as the second parameter (the Minimum limit)
282     SpinnerDateModel checkInModel = new SpinnerDateModel(minDate, minDate, null, java.util.Calendar.DAY_OF_MONTH);
283     SpinnerDateModel checkOutModel = new SpinnerDateModel(minDate, minDate, null, java.util.Calendar.DAY_OF_MONTH);
284
285     checkInSpinner = new JSpinner(checkInModel);
286     checkInSpinner.setBounds(331, 369, 241, 22);
287     checkInSpinner.setEditor(new JSpinner.DateEditor(checkInSpinner, "yyyy-MM-dd"));
288     add(checkInSpinner);
289
290     checkOutSpinner = new JSpinner(checkOutModel);
291     checkOutSpinner.setBounds(331, 390, 241, 22);
292     checkOutSpinner.setEditor(new JSpinner.DateEditor(checkOutSpinner, "yyyy-MM-dd"));
293     add(checkOutSpinner);
294
295     // ADDED: loadAvailableRooms() so changing the date refreshes the dropdown
296     checkInSpinner.addChangeListener(e -> {
297         calculateTotalAmount();
298         loadAvailableRooms();
299     });
300
301     checkOutSpinner.addChangeListener(e -> {
302         calculateTotalAmount();
303         loadAvailableRooms();
304     });
305
306     tvTotalAmount = new JLabel("");

```

```

273     // 1. Get today's date and reset the time to midnight
274     java.util.Calendar today = java.util.Calendar.getInstance();
275     today.set(java.util.Calendar.HOUR_OF_DAY, 0);
276     today.set(java.util.Calendar.MINUTE, 0);
277     today.set(java.util.Calendar.SECOND, 0);
278     today.set(java.util.Calendar.MILLISECOND, 0);
279     Date minDate = today.getTime();
280
281     // 2. Pass minDate as the second parameter (the Minimum limit)
282     SpinnerDateModel checkInModel = new SpinnerDateModel(minDate, minDate, null, java.util.Calendar.DAY_OF_MONTH);
283     SpinnerDateModel checkOutModel = new SpinnerDateModel(minDate, minDate, null, java.util.Calendar.DAY_OF_MONTH);
284
285     checkInSpinner = new JSpinner(checkInModel);
286     checkInSpinner.setBounds(331, 369, 241, 22);
287     checkInSpinner.setEditor(new JSpinner.DateEditor(checkInSpinner, "yyyy-MM-dd"));
288     add(checkInSpinner);
289
290     checkOutSpinner = new JSpinner(checkOutModel);
291     checkOutSpinner.setBounds(331, 390, 241, 22);
292     checkOutSpinner.setEditor(new JSpinner.DateEditor(checkOutSpinner, "yyyy-MM-dd"));
293     add(checkOutSpinner);
294
295     // ADDED: loadAvailableRooms() so changing the date refreshes the dropdown
296     checkInSpinner.addChangeListener(e -> {
297         calculateTotalAmount();
298         loadAvailableRooms();
299     });
300
301     checkOutSpinner.addChangeListener(e -> {
302         calculateTotalAmount();
303         loadAvailableRooms();
304     });
305
306     tvTotalAmount = new JLabel("");

```

```

306     txtTotalAmount = new JTextField();
307     txtTotalAmount.setBounds(331, 500, 241, 21);
308     txtTotalAmount.setEditable(false); // make it read only so they cant edit the price
309     txtTotalAmount.setFont(new Font("SansSerif", Font.BOLD, 14));
310     add(txtTotalAmount);
311
312     // Create an array of our payment option
313     String[] paymentMethods = {
314         "Cash",
315         "GCash/QR Scan"
316     };
317
318     PMethodCBX = new JComboBox(paymentMethods);
319     PMethodCBX.setBounds(331, 470, 241, 26);
320     add(PMethodCBX);
321
322     // 1. Update Clear Form Button
323     JButton btnClearForm = new JButton("Clear Form");
324     btnClearForm.addActionListener(new ActionListener() {
325         public void actionPerformed(ActionEvent e) {
326             clearForm();
327         }
328     });
329     btnClearForm.setForeground(new Color(255, 255, 255));
330     btnClearForm.setBackground(new Color(224, 27, 36));
331     btnClearForm.setFont(new Font("SansSerif", Font.BOLD, 14));
332     // CHANGED: Moved down and right
333     btnClearForm.setBounds(790, 532, 140, 27);
334     add(btnClearForm);
335
336     // 2. Update Save Booking Button
337     JButton btnSaveBooking = new JButton("Save Booking");

```

```

336     // 2. Update Save Booking Button
337     JButton btnSaveBooking = new JButton("Save Booking");
338     btnSaveBooking.addActionListener(new ActionListener() {
339         public void actionPerformed(ActionEvent e) {
340             saveBooking();
341         }
342     });
343     btnSaveBooking.setForeground(new Color(255, 255, 255));
344     btnSaveBooking.setBackground(new Color(46, 194, 126));
345     btnSaveBooking.setFont(new Font("SansSerif", Font.BOLD, 14));
346     // CHANGED: Moved down and right, next to Clear Form
347     btnSaveBooking.setBounds(940, 532, 140, 27);
348     add(btnSaveBooking);
349
350     loadAvailableRooms();
351     // ADDED: para lumabas agad ang default picture ng Standard Room pag bumukas ang panel
352     roomImageLabel.setIcon(createScaledIcon("Standard room.jpg", 815, 649));
353     roomNameLabel.setText("Standard Room");
354
355     //discount checkBox to
356     chkcbxSeniorPwd = new JCheckBox("Senior / PWD");
357     chkcbxSeniorPwd.addActionListener(new ActionListener() {
358         public void actionPerformed(ActionEvent e) {
359             calculateTotalAmount();
360         }
361     });
362     chkcbxSeniorPwd.setBounds(331, 440, 129, 23);
363     add(chkcbxSeniorPwd);
364
365     JLabel lblDiscount = new JLabel("Discount");

```

```

369     lblDiscount.setFont(new Font("SansSerif", Font.BOLD, 14));
370     lblDiscount.setBounds(193, 440, 115, 15);
371     add(lblDiscount);
372
373 }
374
375 //method to load avail rooms depends on selected room type
376 private void loadAvailableRooms() {
377     availableRoomCBX.removeAllItems(); //clear the old list every start
378
379     //this figure out which room type is selected
380     int typeIndex = roomTypeCBX.getSelectedIndex() + 1; //Cuz DB id start at 1
381
382     try {
383         Connection conn = DBConnection.getConnection();
384         String sql = "SELECT room_id, room_number FROM rooms WHERE room_type_id=? AND is_available = 1";
385         PreparedStatement ps = conn.prepareStatement(sql);
386         ps.setInt(1, typeIndex);
387         ResultSet rs = ps.executeQuery();
388
389         boolean found = false;
390
391         while (rs.next()) {
392             availableRoomCBX.addItem(rs.getInt("room_id") + " - Room " + rs.getString("room_number"));
393             found = true;
394         }
395
396         if (!found) availableRoomCBX.addItem("No rooms available");
397     } catch (SQLException e) {
398         JOptionPane.showMessageDialog(this, "Error loading rooms: " + e.getMessage());
399     }
400 }
401
402 private void calculateTotalAmount() {

```

```

402 private void calculateTotalAmount() {
403     try {
404         // get dates from spinner
405         Date checkInDate = (Date) checkInSpinner.getValue();
406         Date checkOutDate = (Date) checkOutSpinner.getValue();
407
408         // Strip the TIME part - keep only the date
409         // This resets hours, minutes, seconds to 00:00:00
410         java.util.Calendar calIn = java.util.Calendar.getInstance();
411         calIn.setTime(checkInDate);
412         calIn.set(java.util.Calendar.HOUR_OF_DAY, 0);
413         calIn.set(java.util.Calendar.MINUTE, 0);
414         calIn.set(java.util.Calendar.SECOND, 0);
415         calIn.set(java.util.Calendar.MILLISECOND, 0);
416
417         java.util.Calendar calOut = java.util.Calendar.getInstance();
418         calOut.setTime(checkOutDate);
419         calOut.set(java.util.Calendar.HOUR_OF_DAY, 0);
420         calOut.set(java.util.Calendar.MINUTE, 0);
421         calOut.set(java.util.Calendar.SECOND, 0);
422         calOut.set(java.util.Calendar.MILLISECOND, 0);
423
424         // now calculate days cleanly
425         long diffInMillis = calOut.getTimeInMillis() - calIn.getTimeInMillis();
426         long days = diffInMillis / (1000 * 60 * 60 * 24);
427
428         if (days <= 0) {
429             txtTotalAmount.setText("Invalid Dates");
430             lblExtraCharge.setText("");
431             return;
432         }
433
434         double[] basePrices = {2500.0, 3500.0, 5000.0, 8000.0};

```

```

435         int[] includedGuests = {2, 4, 6, 10}; // Base capacity
436         double[] extraGuestFee = {500.0, 700.0, 1000.0, 1500.0}; // Fee per extra head
437
438         int typeIndex = roomTypeCBX.getSelectedIndex();
439         int actualGuests = (int) guestCountSpinner.getValue();
440         int allowedBaseGuests = includedGuests[typeIndex];
441
442         // Calculate Base Room Cost
443         double total = days * basePrices[typeIndex];
444
445
446
447         // Calculate Extra Guest Surcharge
448         int extraGuests = Math.max(0, actualGuests - allowedBaseGuests);
449         double extraChargeTotal = extraGuests * extraGuestFee[typeIndex] * days;
450
451         // then we will now display how much is the total charged
452         if (extraGuests > 0) {
453             lblExtraCharge.setText(String.format("(+ P%,.2f for %d extra guests)", extraChargeTotal, extraGuests));
454         } else {
455             lblExtraCharge.setText(""); // clear if no extra guests
456         }
457
458         // Finalize Total
459         total += extraChargeTotal;
460         total = applyDiscount(total); // Apply Senior/PwD if checked
461
462         txtTotalAmount.setText("P" + String.format("%.2f", total));
463
464
465     } catch (Exception e) {
466         txtTotalAmount.setText("Error");
467     }
468

```

```

468     }
469
470 private void clearForm() {
471     FNameField.setText("");
472     LNameField.setText("");
473     PNumField.setText("");
474     emailField.setText("");
475     roomTypeCBX.setSelectedIndex(0);
476     PMethodCBX.setSelectedIndex(0);
477     checkInSpinner.setValue(new Date());
478     checkOutSpinner.setValue(new Date());
479     txtTotalAmount.setText("P 0.00");
480     // ADDED: i-clear ang image at label pag na-clear ang form
481     roomImageLabel.setIcon(null);
482     roomNameLabel.setText("");
483     // ADDED: i-reset ang guest count spinner sa 1
484     guestCountSpinner.setValue(1);
485     chkcbxSeniorPwD.setSelected(false); // reverse the check box to unchecked
486     lblExtraCharge.setText("");
487     roomTypeCBX.setSelectedIndex(0);
488 }
489
490
491
492
493
494
495 private void saveBooking() {
496     // STEP 1 - Validate empty fields
497     if (FNameField.getText().trim().isEmpty() || LNameField.getText().trim().isEmpty()
498         || PNumField.getText().trim().isEmpty() || emailField.getText().trim().isEmpty()) {
499         JOptionPane.showMessageDialog(this, "Please fill in all customer information.", "Validation Error", JOptionPane.WARNING_MESSAGE);
500         return;
501     }

```



```

501     }
502
503     // STEP 2 - Validate phone number (must be 10-11 digits)
504     String phone = PNumField.getText().trim();
505     if (!phone.matches("\\d{10,11}")) {
506         JOptionPane.showMessageDialog(this, "Please enter a valid phone number (10-11 digits).", "Validation Error", JOptionPane.WARNING_MESSAGE);
507         return;
508     }
509
510     // STEP 3 - Validate email
511     String email = emailField.getText().trim();
512     if (!email.contains("@") || !email.contains(".")) {
513         JOptionPane.showMessageDialog(this, "Please enter a valid email address.", "Validation Error", JOptionPane.WARNING_MESSAGE);
514         return;
515     }
516
517     // BLOCK booking if customer already has an active booking (Reserved OR Checked In)
518     if (currentCustomerId != -1) {
519         try (Connection conn = DBConnection.getConnection()) {
520             String checkSQL = "SELECT b.booking_id, r.room_number, b.status FROM bookings b " +
521                             "JOIN rooms r ON b.room_id = r.room_id " +
522                             "WHERE b.customer_id = ? AND b.status IN ('Reserved', 'Checked In')";
523             PreparedStatement checkPS = conn.prepareStatement(checkSQL);
524             checkPS.setInt(1, currentCustomerId);
525             ResultSet checkRS = checkPS.executeQuery();
526
527             if (checkRS.next()) {
528                 String roomNum = checkRS.getString("room_number");
529                 String status = checkRS.getString("status");
530                 JOptionPane.showMessageDialog(this,
531                     "This customer already has an active booking at Room " + roomNum + " (Status: " + status + ").\n" +
532                     "Cannot create a new booking while a reservation or check-in is active.",
533                     "Booking Blocked",
534                     JOptionPane.WARNING_MESSAGE);

```

```

535             }
536             return; // STOP - do not save
537         } catch (SQLException ex) {
538             JOptionPane.showMessageDialog(this, "Error checking customer status: " + ex.getMessage());
539             return;
540         }
541     }
542
543     // ADDED: kunin ang value ng guest count spinner
544     int guestCount = (int) guestCountSpinner.getValue();
545
546     // ADDED: i-check kung hindi lumagpas sa limit ng room type
547     // Standard=3, Junior=6, Executives=10, Presidential=14
548     int[] guestLimits = {3, 6, 10, 14};
549     int limit = guestLimits[roomTypeCBX.getSelectedIndex()];
550
551     if (guestCount > limit) {
552         JOptionPane.showMessageDialog(this,
553             "Too many guests! Maximum for this room is " + limit + " guests.",
554             "Validation Error", JOptionPane.WARNING_MESSAGE);
555         return;
556     }
557
558     // STEP 4 - Validate room selection
559     String roomEntry = (String) availableRoomCBX.getSelectedItem();
560     if (roomEntry == null || roomEntry.startsWith("No rooms")) {
561         JOptionPane.showMessageDialog(this, "No available room selected.", "Validation Error", JOptionPane.WARNING_MESSAGE);
562         return;
563     }
564
565     // STEP 4b - Safety: double-check room is still actually available
566     int roomId = Integer.parseInt(roomEntry.split(" - ")[0].trim());

```

```

567     try (Connection conn = DBConnection.getConnection()) {
568         PreparedStatement ps = conn.prepareStatement("SELECT is_available FROM rooms WHERE room_id = ?");
569         ps.setInt(1, roomId);
570         ResultSet rs = ps.executeQuery();
571         if (rs.next() && rs.getInt("is_available") == 0) {
572             JOptionPane.showMessageDialog(this,
573                 "Room " + roomEntry + " was just taken by another booking.\nPlease select a different room.",
574                 "Room No Longer Available", JOptionPane.WARNING_MESSAGE);
575             loadAvailableRooms();
576             return;
577         }
578     } catch (SQLException ex) {
579         JOptionPane.showMessageDialog(this, "Error checking room availability: " + ex.getMessage());
580         return;
581     }
582
583     // STEP 5 - Get dates and strip the time part
584     Date checkIn = (Date) checkInSpinner.getValue();
585     Date checkOut = (Date) checkOutSpinner.getValue();
586
587     java.util.Calendar calIn = java.util.Calendar.getInstance();
588     calIn.setTime(checkIn);
589     calIn.set(java.util.Calendar.HOUR_OF_DAY, 0);
590     calIn.set(java.util.Calendar.MINUTE, 0);
591     calIn.set(java.util.Calendar.SECOND, 0);
592     calIn.set(java.util.Calendar.MILLISECOND, 0);
593
594     java.util.Calendar calOut = java.util.Calendar.getInstance();
595     calOut.setTime(checkOut);
596     calOut.set(java.util.Calendar.HOUR_OF_DAY, 0);
597     calOut.set(java.util.Calendar.MINUTE, 0);
598     calOut.set(java.util.Calendar.SECOND, 0);

```

```

600     calOut.set(java.util.Calendar.SECOND, 0);
601     calOut.set(java.util.Calendar.MILLISECOND, 0);
602
603     // STEP 6 - Validate dates
604     long days = (calOut.getTimeInMillis() - calIn.getTimeInMillis()) / (1000 * 60 * 60 * 24);
605     if (days <= 0) {
606         JOptionPane.showMessageDialog(this, "Check-out must be after check-in.", "Validation Error", JOptionPane.WARNING_MESSAGE);
607         return;
608     }
609
610     // use stripped dates for DB
611     checkIn = calIn.getTime();
612     checkOut = calOut.getTime();
613
614
615
616     // STEP 7 - Calculate total for Database
617
618     double[] basePrices = {2500.0, 3500.0, 5000.0, 8000.0};
619     int[] includedGuests = {2, 4, 6, 10};
620     double[] extraGuestFee = {500.0, 700.0, 1000.0, 1500.0};
621
622
623
624     int typeIndex = roomTypeCBX.getSelectedIndex();
625     int actualGuests = (int) guestCountSpinner.getValue();
626     int allowedBaseGuests = includedGuests[typeIndex];
627     int extraGuests = Math.max(0, actualGuests - allowedBaseGuests);
628
629     double baseTotal = basePrices[typeIndex] * days;
630     double extraChargeTotal = extraGuests * extraGuestFee[typeIndex] * days;
631
632     double total = baseTotal + extraChargeTotal;
633

```

```

633
634     //discount if its applicable
635     total = applyDiscount(total);
636
637
638     // STEP 8 - Convert to SQL dates
639     java.sql.Date sqlCheckIn = new java.sql.Date(checkIn.getTime());
640     java.sql.Date sqlCheckOut = new java.sql.Date(checkOut.getTime());
641     String payment = (String) PMethodCBX.getSelectedItem();
642
643     // STEP 9 - Save to database
644     try (Connection conn = DBConnection.getConnection()) {
645         conn.setAutoCommit(false); // start transaction
646
647         try {
648             int finalCustomerId = currentCustomerId;
649
650             // 1. ONLY insert a new customer if they are brand new (ID is -1)
651             if (finalCustomerId == -1) {
652                 // Brand new customer -> INSERT
653                 String custSQL = "INSERT INTO customers (first_name, last_name, phone_number, email, created_at) VALUES (?, ?, ?, ?, NOW())";
654                 PreparedStatement custPS = conn.prepareStatement(custSQL, Statement.RETURN_GENERATED_KEYS);
655                 custPS.setString(1, FNameField.getText().trim());
656                 custPS.setString(2, LNameField.getText().trim());
657                 custPS.setString(3, phone);
658                 custPS.setString(4, email);
659                 custPS.executeUpdate();
660
661                 ResultSet keys = custPS.getGeneratedKeys();
662                 if (keys.next()) finalCustomerId = keys.getInt(1);
663             }
664             else {
665                 // EXISTING customer -> UPDATE their contact info just in case they changed it on the form!
666                 String updateCNI = "UPDATE customers SET phone_number = ?, email = ? WHERE customer_id = ?";

```

```

666
667                 String updateSQL = "UPDATE customers SET phone_number = ?, email = ? WHERE customer_id = ?";
668                 PreparedStatement updatePS = conn.prepareStatement(updateSQL);
669                 updatePS.setString(1, phone);
670                 updatePS.setString(2, email);
671                 updatePS.setInt(3, finalCustomerId);
672                 updatePS.executeUpdate();
673             }
674
675             // 2. Insert booking (CHANGED: 'Active' is now perfectly set to 'Reserved')
676             String bookSQL = "INSERT INTO bookings (customer_id, room_id, check_in_date, check_out_date, total_amount, payment_method, status) VALUES";
677             PreparedStatement bookPS = conn.prepareStatement(bookSQL);
678             bookPS.setInt(1, finalCustomerId);
679             bookPS.setInt(2, roomId);
680             bookPS.setDate(3, sqlCheckIn);
681             bookPS.setDate(4, sqlCheckOut);
682             bookPS.setDouble(5, total);
683             bookPS.setString(6, payment);
684             bookPS.executeUpdate();
685
686             // 3. Mark room as occupied
687             PreparedStatement roomPS = conn.prepareStatement("UPDATE rooms SET is_available = 0 WHERE room_id = ?");
688             roomPS.setInt(1, roomId);
689             roomPS.executeUpdate();
690
691             conn.commit(); // save everything!
692
693             // Success message
694             JOptionPane.showMessageDialog(this,
695                 String.format("☑ Booking saved!\nCustomer: %s %s\nTotal: P%.2f",
696                     FNameField.getText(), LNameField.getText(), total),
697                 "Booking Confirmed", JOptionPane.INFORMATION_MESSAGE);
698
699             clearForm();
700

```

```

699         loadAvailableRooms();
700
701     } catch (SQLException ex) {
702         conn.rollback(); // undo everything if error!
703         throw ex;
704     } finally {
705         conn.setAutoCommit(true);
706     }
707
708     } catch (Exception ex) {
709         JOptionPane.showMessageDialog(this, "Database error: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
710     }
711 }
712
713
714
715
716
717
718 // ADDED: helper method para i-resize ang image
719 private ImageIcon createScaledIcon(String path, int width, int height) {
720     try {
721         ImageIcon icon = new ImageIcon(path);
722         if (icon.getWidth() == -1) {
723             System.out.println("Warning: Could not find image at " + path);
724             return null;
725         }
726         Image img = icon.getImage();
727         Image newImg = img.getScaledInstance(width, height, Image.SCALE_SMOOTH);
728         return new ImageIcon(newImg);
729     } catch (Exception e) {
730         return null;
731     }
732 }

```

```

732 }
733
734
735
736 //discount method
737 private double applyDiscount(double subTotal) {
738     if (chckbxSeniorPwv.isSelected()) {
739         return subTotal - (subTotal * 0.20);
740     }
741     return subTotal;
742 }
743
744 //search customer method
745 private void showCustomerSearchDialog() {
746     JDialog searchDialog = new JDialog((Frame) SwingUtilities.getWindowAncestor(this), "Search Customer", true);
747     searchDialog.setSize(500, 350);
748     searchDialog.setLocationRelativeTo(this);
749     searchDialog.getContentPane().setLayout(new BorderLayout());
750
751     // Top Panel: Search Bar
752     JPanel topPanel = new JPanel(new FlowLayout());
753     JTextField txtSearch = new JTextField(20);
754     JButton btnSearch = new JButton("Search");
755     topPanel.add(new JLabel("Name:"));
756     topPanel.add(txtSearch);
757     topPanel.add(btnSearch);
758     searchDialog.getContentPane().add(topPanel, BorderLayout.NORTH);
759
760     txtSearch.addKeyListener(new java.awt.event.KeyAdapter() {
761         public void keyPressed(java.awt.event.KeyEvent e) {
762             char c = e.getKeyChar();
763             // block numbers - hindi makakapag type ng number
764             if (c >= '0' && c <= '9') {

```

```

765                 e.consume();
766             }
767             // auto uppercase - kahit hindi pinindot caps lock
768             if (Character.isLetter(c)) {
769                 e.setKeyChar(Character.toUpperCase(c));
770             }
771         }
772     });
773
774
775     // Center Panel: Results Table
776     String[] columns = {"ID", "First Name", "Last Name", "Phone"};
777     javax.swing.table.DefaultTableModel model = new javax.swing.table.DefaultTableModel(columns, 0) {
778         public boolean isCellEditable(int row, int column) { return false; }
779     };
780     JTable table = new JTable(model);
781     searchDialog.getContentPane().add(new JScrollPane(table), BorderLayout.CENTER);
782
783     // Bottom Panel: Action Buttons
784     JPanel bottomPanel = new JPanel();
785     JButton btnSelect = new JButton("Select Customer");
786     bottomPanel.add(btnSelect);
787     searchDialog.getContentPane().add(bottomPanel, BorderLayout.SOUTH);
788
789     // The Search Action (Database Query)
790     btnSearch.addActionListener(e -> {
791         model.setRowCount(0); // clear previous results
792         String keyword = "%" + txtSearch.getText().trim() + "%";
793
794         try (Connection conn = DBConnection.getConnection()) {
795             String sql = "SELECT customer_id, first_name, last_name, phone_number, email FROM customers WHERE first_name LIKE ? OR last_name LIKE ?";
796             PreparedStatement ps = conn.prepareStatement(sql);
797             ps.setString(1, keyword);
798             ps.setString(2, keyword);

```

```

797         ps.setString(2, keyword);
798     }
799     ResultSet rs = ps.executeQuery();
800
801     while (rs.next()) {
802         model.addRow(new Object[]{
803             rs.getInt("customer_id"),
804             rs.getString("first_name"),
805             rs.getString("last_name"),
806             rs.getString("phone_number")
807         });
808     }
809 } catch (SQLException ex) {
810     JOptionPane.showMessageDialog(searchDialog, "Search Error: " + ex.getMessage());
811 }
812 });
813
814 // The Select Action (Auto-fill the main form)
815 btnSelect.addActionListener(e -> {
816     int selectedRow = table.getSelectedRow();
817     if (selectedRow != -1) {
818         // 1. Save the ID
819         currentCustomerId = (int) model.getValueAt(selectedRow, 0);
820
821         // 2. Fetch full details to populate form
822         try (Connection conn = DBConnection.getConnection()) {
823             PreparedStatement ps = conn.prepareStatement("SELECT * FROM customers WHERE customer_id = ?");
824             ps.setInt(1, currentCustomerId);
825             ResultSet rs = ps.executeQuery();
826             if (rs.next()) {
827                 FNameField.setText(rs.getString("first_name"));
828                 LNameField.setText(rs.getString("last_name"));
829                 PNumField.setText(rs.getString("phone_number"));
830                 emailField.setText(rs.getString("email"));
831             }
832         }
833     }
834 }

```

```

814 // The Select Action (Auto-fill the main form)
815 btnSelect.addActionListener(e -> {
816     int selectedRow = table.getSelectedRow();
817     if (selectedRow != -1) {
818         // 1. Save the ID
819         currentCustomerId = (int) model.getValueAt(selectedRow, 0);
820
821         // 2. Fetch full details to populate form
822         try (Connection conn = DBConnection.getConnection()) {
823             PreparedStatement ps = conn.prepareStatement("SELECT * FROM customers WHERE customer_id = ?");
824             ps.setInt(1, currentCustomerId);
825             ResultSet rs = ps.executeQuery();
826             if (rs.next()) {
827                 FNameField.setText(rs.getString("first_name"));
828                 LNameField.setText(rs.getString("last_name"));
829                 PNumField.setText(rs.getString("phone_number"));
830                 emailField.setText(rs.getString("email"));
831             }
832             // 3. Lock fields to prevent editing an existing DB record
833             FNameField.setEditable(false);
834             LNameField.setEditable(false);
835             PNumField.setEditable(true);
836             emailField.setEditable(true);
837         }
838     } catch (SQLException ex) {
839         ex.printStackTrace();
840     }
841     searchDialog.dispose(); // Close popup
842 }
843 });
844
845 searchDialog.setVisible(true);
846 }
847 }

```

5.MANAGE BOOKING PANEL

```

13 import javax.swing.*;
14
15 public class ManageBookingPanel extends JPanel {
16     private static final long serialVersionUID = 1L;
17     private JTable table;
18     private DefaultTableModel tableModel;
19
20     public ManageBookingPanel() {
21         setLayout(new BorderLayout());
22
23         // --- Header ---
24         JLabel titleLabel = new JLabel("Manage Booking");
25         titleLabel.setForeground(new Color(0, 0, 47));
26         titleLabel.setBackground(new Color(240, 240, 240));
27         titleLabel.setOpaque(true);
28         titleLabel.setFont(new Font("SansSerif", Font.BOLD, 20));
29         titleLabel.setBorder(new EmptyBorder(15, 15, 15, 0));
30         add(titleLabel, BorderLayout.NORTH);
31
32         // --- Table ---
33         String[] columns = {"Booking ID", "Customer Name", "Room No.", "Room Type", "Check-In", "Check-Out", "Total (P)", "Payment", "Status"};
34         tableModel = new DefaultTableModel(columns, 0) {
35             @Override
36             public boolean isCellEditable(int row, int col) { return false; }
37         };
38
39         table = new JTable(tableModel);
40         table.setRowHeight(28);
41         table.setFont(new Font("SansSerif", Font.PLAIN, 13));
42         table.getTableHeader().setFont(new Font("SansSerif", Font.BOLD, 13));
43         table.getTableHeader().setForeground(Color.WHITE);
44         table.getTableHeader().setBackground(new Color(31, 71, 145));
45
46         JScrollPane scrollPane = new JScrollPane();
47         add(scrollPane, BorderLayout.CENTER);

```

```

47 add(scrollPane, BorderLayout.CENTER);
48 scrollPane.setViewportView(table);
49
50 // --- Action Buttons ---
51 JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 10));
52 add(buttonPanel, BorderLayout.SOUTH);
53
54 JButton btnRefresh = new JButton("Refresh");
55 JButton btnCheckIn = new JButton("Check In");
56 JButton btnCheckOut = new JButton("Check Out");
57 JButton btnExtend = new JButton("Extend Stay");
58 JButton btnUpgrade = new JButton("Upgrade Room");
59 JButton btnReschedule = new JButton("Reschedule");
60 JButton btnCancel = new JButton("Cancel Booking");
61
62 // Styling Buttons
63 btnCheckIn.setBackground(new Color(46, 194, 126)); btnCheckIn.setForeground(Color.WHITE);
64 btnCheckOut.setBackground(new Color(255, 165, 0)); btnCheckOut.setForeground(Color.WHITE);
65 btnExtend.setBackground(new Color(148, 0, 211)); btnExtend.setForeground(Color.WHITE);
66 btnUpgrade.setBackground(new Color(32, 178, 170)); btnUpgrade.setForeground(Color.WHITE);
67 btnReschedule.setBackground(new Color(0, 128, 255)); btnReschedule.setForeground(Color.WHITE);
68 btnCancel.setBackground(new Color(224, 27, 36)); btnCancel.setForeground(Color.WHITE);
69
70 buttonPanel.add(btnRefresh);
71 buttonPanel.add(btnCheckIn);
72 buttonPanel.add(btnCheckOut);
73 buttonPanel.add(btnExtend);
74 buttonPanel.add(btnUpgrade);
75 buttonPanel.add(btnReschedule);
76 buttonPanel.add(btnCancel);
77
78 // Listeners
79 btnRefresh.addActionListener(e -> loadBookings());
80 btnCheckIn.addActionListener(e -> checkInBooking());
81 btnCheckOut.addActionListener(e -> checkOutBooking());

```

```

82 btnExtend.addActionListener(e -> extendStayBooking());
83 btnUpgrade.addActionListener(e -> upgradeRoomBooking());
84 btnReschedule.addActionListener(e -> rescheduleBooking());
85 btnCancel.addActionListener(e -> cancelBooking());
86
87 loadBookings();
88 }
89
90 // =====
91 // 1. Load Bookings & Auto-Update Rules
92 // =====
93 private void loadBookings() {
94     tableModel.setRowCount(0);
95     try (Connection conn = DBConnection.getConnection()) {
96
97         // Auto Check-Out guests if their date has passed
98         Statement autoStmnt = conn.createStatement();
99         autoStmnt.executeUpdate("UPDATE bookings SET status = 'Checked Out' WHERE check_out_date < CURDATE() AND status = 'Checked In'");
100
101         // Free rooms ONLY if they have NO active bookings (Reserved or Checked In)
102         Statement autoFreeStmnt = conn.createStatement();
103         autoFreeStmnt.executeUpdate(
104             "UPDATE rooms r SET r.is_available = 1 " +
105             "WHERE r.is_available = 0 " +
106             "AND NOT EXISTS ( " +
107             "    SELECT 1 FROM bookings b " +
108             "    WHERE b.room_id = r.room_id " +
109             "    AND b.status IN ('Reserved', 'Checked In') " +
110             ") "
111         );
112
113         String sql = "SELECT b.booking_id, CONCAT(c.first_name, ' ', c.last_name) AS customer, r.room_number, rt.type_name, b.check_in_date, b.check_out_date";
114         ResultSet rs = conn.createStatement().executeQuery(sql);
115
116         while (rs.next()) {
117             // Add row to table model

```

```

118             tableModel.addRow(new Object[] {
119                 rs.getInt("booking_id"), rs.getString("customer"), rs.getString("room_number"),
120                 rs.getString("type_name"), rs.getString("check_in_date"), rs.getString("check_out_date"),
121                 rs.getDouble("total_amount"), rs.getString("payment_method"), rs.getString("status")
122             });
123         } catch (SQLException e) {
124             JOptionPane.showMessageDialog(this, "Error loading bookings: " + e.getMessage());
125         }
126     }
127
128 // =====
129 // 2. Check In
130 // =====
131 private void checkInBooking() {
132     int row = table.getSelectedRow();
133     if (row == -1) return;
134     if (!tableModel.getValueAt(row, 8).equals("Reserved")) {
135         JOptionPane.showMessageDialog(this, "Only 'Reserved' bookings can be Checked In."); return;
136     }
137     try (Connection conn = DBConnection.getConnection()) {
138         PreparedStatement ps = conn.prepareStatement("UPDATE bookings SET status = 'Checked In' WHERE booking_id = ?");
139         ps.setInt(1, (int) tableModel.getValueAt(row, 0));
140         ps.executeUpdate();
141         loadBookings();
142     } catch (SQLException e) { e.printStackTrace(); }
143 }
144
145 // =====
146 // 3. Check Out
147 // =====
148 private void checkOutBooking() {
149     int row = table.getSelectedRow();
150     if (row == -1) return;
151     if (!tableModel.getValueAt(row, 8).equals("Checked In")) {

```



```

256         availableRoomCombo.removeAllItems();
257         roomIds.clear();
258         roomRates.clear();
259
260         int typeId = roomTypeCombo.getSelectedIndex() + 1; // Database IDs start at 1
261
262         try (Connection conn = DBConnection.getConnection()) {
263             PreparedStatement ps = conn.prepareStatement(
264                 "SELECT r.room_id, r.room_number, rt.rate_per_day " +
265                 "FROM rooms r JOIN room_types rt ON r.room_type_id = rt.room_type_id " +
266                 "WHERE r.room_type_id = ? AND r.is_available = 1"
267             );
268             ps.setInt(1, typeId);
269             ResultSet rs = ps.executeQuery();
270
271             boolean found = false;
272             while (rs.next()) {
273                 availableRoomCombo.addItem("Room " + rs.getString("room_number"));
274                 roomIds.add(rs.getInt("room_id"));
275                 roomRates.add(rs.getDouble("rate_per_day"));
276                 found = true;
277             }
278             if (!found) availableRoomCombo.addItem("No rooms available");
279         } catch (SQLException ex) {
280             ex.printStackTrace();
281         }
282     }
283 }
284
285 // Hook up the listener and trigger it once to load the initial list
286 roomTypeCombo.addActionListener(updateRoomsAction);
287 updateRoomsAction.actionPerformed(null);
288
289 // --- 3. Show the Popup ---
290 if (JOptionPane.showConfirmDialog(this, panel, "Upgrade Room", JOptionPane.OK_CANCEL_OPTION) == JOptionPane.OK_OPTION) {

```

```

291     String selectedRoom = (String) availableRoomCombo.getSelectedItem();
292     if (selectedRoom == null || selectedRoom.equals("No rooms available")) {
293         JOptionPane.showMessageDialog(this, "Invalid room selection. Upgrade cancelled.", "Error", JOptionPane.ERROR_MESSAGE);
294         return;
295     }
296
297     // Get data for the selected room
298     int selectedIndex = availableRoomCombo.getSelectedIndex();
299     int newRoomId = roomIds.get(selectedIndex);
300     double newRate = roomRates.get(selectedIndex);
301
302     // Calculate new totals
303     double oldRate = getRateForType(oldRoomType);
304     double diff = (newRate - oldRate) * days;
305     double newTotal = currentTotal + diff;
306
307     // --- 4. Final Confirmation ---
308     if (JOptionPane.showConfirmDialog(this, "Price Difference: P" + String.format("%.2f", diff) + "\nNew Total: P" + String.format("%.2f", newTotal),
309         "Confirm Upgrade", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
310         try (Connection conn = DBConnection.getConnection()) {
311             conn.setAutoCommit(false);
312
313             // Update booking
314             PreparedStatement ps = conn.prepareStatement("UPDATE bookings SET room_id = ?, total_amount = ? WHERE booking_id = ?");
315             ps.setInt(1, newRoomId);
316             ps.setDouble(2, newTotal);
317             ps.setInt(3, bookingId);
318             ps.executeUpdate();
319
320             // Free the old room and occupy the new room
321             conn.prepareStatement("UPDATE rooms SET is_available = 1 WHERE room_number = " + oldRoomNum + ";").executeUpdate();
322             conn.prepareStatement("UPDATE rooms SET is_available = 0 WHERE room_id = " + newRoomId).executeUpdate();
323
324             conn.commit();
325             JOptionPane.showMessageDialog(this, "Room upgraded successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
326             loadBookings(); // Refresh the table

```

```

326         loadBookings(); // Refresh the table
327     }
328 }
329 } catch (Exception e) {
330     e.printStackTrace();
331 }
332 }
333 }
334
335 // =====
336 // 6. Reschedule
337 // =====
338 // in resched lalabas lng is ung exact date na binook mo dun sa spinner date ng resched at dto nareverse ng past date, so hindi na pwedeng i-atras ang ch
339 private void rescheduleBooking() {
340     int row = table.getSelectedRow();
341     if (row == -1) return;
342     if (tableModel.getValueAt(row, 8).equals("Reserved")) {
343         JOptionPane.showMessageDialog(this, "Only 'Reserved' bookings can be rescheduled."); return;
344     }
345
346     int bookingId = (int) tableModel.getValueAt(row, 0);
347     double dailyRate = getRateForType((String) tableModel.getValueAt(row, 3));
348
349     try {
350         // Kunin ang original dates mula sa table (columns 4 at 5)
351         SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
352         Date originalCheckIn = sdf.parse((String) tableModel.getValueAt(row, 4));
353         Date originalCheckOut = sdf.parse((String) tableModel.getValueAt(row, 5));
354
355         // I-set ang minimum ng check-in sa original check-in date
356         // Hindi na pwedeng i-atras ang check-in date
357         SpinnerDateModel checkInModel = new SpinnerDateModel(originalCheckIn, originalCheckIn, null, Calendar.DAY_OF_MONTH);
358         // I-set ang initial value ng check-out sa original check-out date
359         SpinnerDateModel checkOutModel = new SpinnerDateModel(originalCheckOut, originalCheckIn, null, Calendar.DAY_OF_MONTH);
360     }

```

```

361 JSpinner checkInSpinner = new JSpinner(checkInModel);
362 JSpinner checkOutSpinner = new JSpinner(checkOutModel);
363 checkInSpinner.setEditor(new JSpinner.DateEditor(checkInSpinner, "yyyy-MM-dd"));
364 checkOutSpinner.setEditor(new JSpinner.DateEditor(checkOutSpinner, "yyyy-MM-dd"));
365
366 JPanel datePanel = new JPanel(new GridLayout(2, 2, 5, 5));
367 datePanel.add(new JLabel("New Check-In Date:")); datePanel.add(checkInSpinner);
368 datePanel.add(new JLabel("New Check-Out Date:")); datePanel.add(checkOutSpinner);
369
370 if (JOptionPane.showConfirmDialog(this, datePanel, "Pick New Dates", JOptionPane.OK_CANCEL_OPTION) == JOptionPane.OK_OPTION) {
371     Date in = sdf.parse(sdf.format(checkInSpinner.getValue()));
372     Date out = sdf.parse(sdf.format(checkOutSpinner.getValue()));
373
374     long days = (out.getTime() - in.getTime()) / (1000 * 60 * 60 * 24);
375     if (days <= 0) { JOptionPane.showMessageDialog(this, "Check-out must be after check-in."); return; }
376
377     double fee = dailyRate * 0.15;
378     double newTotal = (dailyRate * days) + fee;
379
380     if (JOptionPane.showConfirmDialog(this, String.format("Reschedule Fee: P%,.2f\nNew Total: P%,.2f\nConfirm?", fee, newTotal), "Confirm", JOptionPane.OK_CANCEL_OPTION) == JOptionPane.OK_OPTION) {
381         try (Connection conn = DBConnection.getConnection()) {
382             PreparedStatement ps = conn.prepareStatement("UPDATE bookings SET check_in_date=?, check_out_date=?, total_amount=?, reschedule_charge=?");
383             ps.setString(1, sdf.format(in)); ps.setString(2, sdf.format(out));
384             ps.setDouble(3, newTotal); ps.setDouble(4, fee); ps.setInt(5, bookingId);
385             ps.executeUpdate();
386             loadBookings();
387         }
388     }
389 } catch (Exception e) { e.printStackTrace(); }
390
391 // =====
392 // 7. Cancel
393 // =====
394 private void cancelBooking() {

```

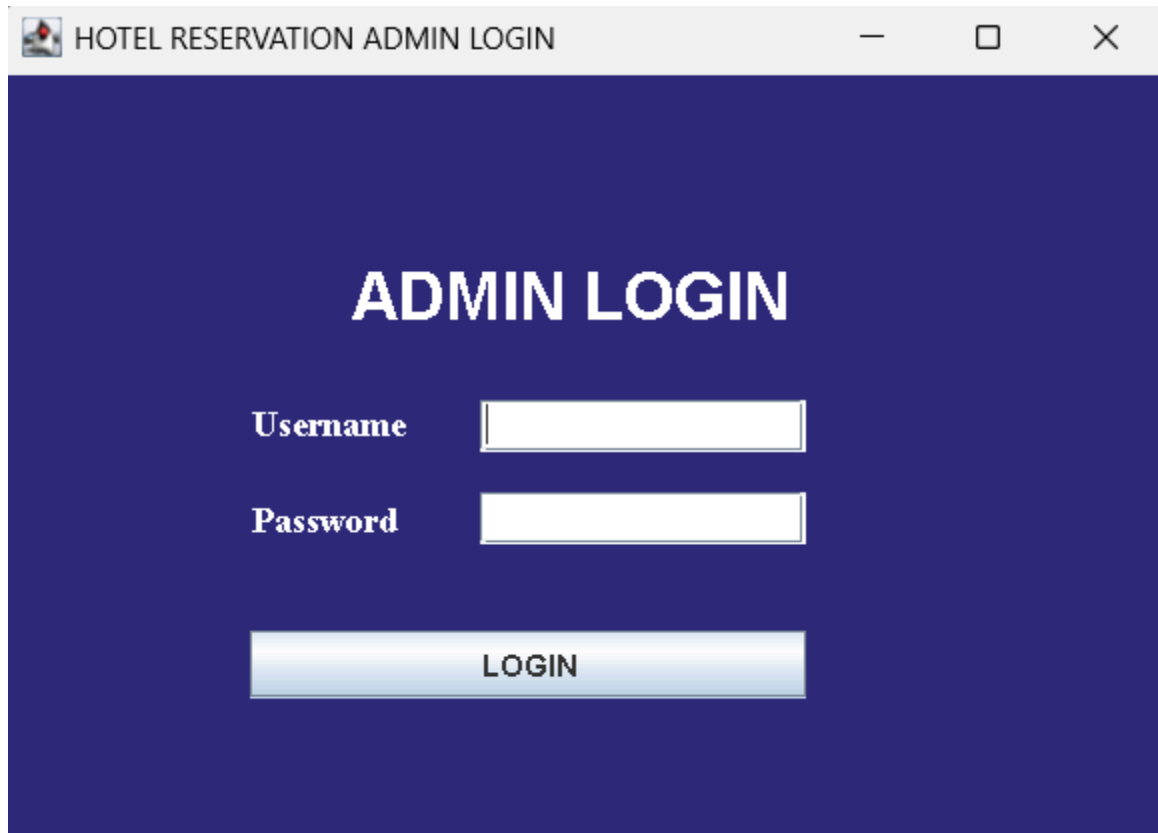
```

389 } catch (Exception e) { e.printStackTrace(); }
390
391 // =====
392 // 7. Cancel
393 // =====
394 private void cancelBooking() {
395     int row = table.getSelectedRow();
396     if (row == -1) return;
397     if (!tableModel.getValueAt(row, 8).equals("Reserved")) {
398         JOptionPane.showMessageDialog(this, "Only 'Reserved' bookings can be cancelled."); return;
399     }
400
401     try (Connection conn = DBConnection.getConnection()) {
402         int bookingId = (int) tableModel.getValueAt(row, 0);
403         conn.prepareStatement("UPDATE bookings SET status = 'Cancelled' WHERE booking_id = " + bookingId).executeUpdate();
404         conn.prepareStatement("UPDATE rooms r JOIN bookings b ON b.room_id = r.room_id SET r.is_available = 1 WHERE b.booking_id = " + bookingId).executeUpdate();
405         loadBookings();
406     } catch (SQLException e) { e.printStackTrace(); }
407
408 // =====
409 // Helper: Get Rate
410 // =====
411 private double getRateForType(String typeName) {
412     try (Connection conn = DBConnection.getConnection()) {
413         PreparedStatement ps = conn.prepareStatement("SELECT rate_per_day FROM room_types WHERE type_name = ?");
414         ps.setString(1, typeName);
415         ResultSet rs = ps.executeQuery();
416         if (rs.next()) return rs.getDouble("rate_per_day");
417     } catch (SQLException e) { e.printStackTrace(); }
418     return 0.0;
419 }
420
421 }
422
423

```


Sample output

LOGIN FRAME



The image shows a web browser window titled "HOTEL RESERVATION ADMIN LOGIN". The window has a dark blue background. In the center, the text "ADMIN LOGIN" is displayed in large, white, bold, sans-serif capital letters. Below this text, there are two input fields. The first field is labeled "Username" in white text to its left. The second field is labeled "Password" in white text to its left. Both fields are white with a thin blue border. Below the input fields, there is a large, light blue button with a gradient and the word "LOGIN" in black, bold, sans-serif capital letters. The window's title bar is light gray and contains a small icon on the left and standard minimize, maximize, and close buttons on the right.

HOTEL RESERVATION ADMIN LOGIN

ADMIN LOGIN

Username

Password

LOGIN

DashBoard Frame



The image shows the top header of a dashboard. It has a dark blue background. On the left, the text "Sync Suites Hotel" is displayed in white, sans-serif font. On the right, there is a red button with the word "Logout" in white, sans-serif font. Below the header, there is a light gray bar containing three tabs: "New Booking", "Room Availability", and "Manage Booking". Each tab is a small rectangle with its label in a dark gray font.

Sync Suites Hotel

Logout

New Booking Room Availability Manage Booking

New Booking Panel

Sync Suites Hotel

Logout

New Booking

Room Availability

Manage Booking

CREATE NEW BOOKING

--Customer Information

Search Customer

First Name:

Last Name:

Phone Number:

Email:

--Room Selection

Room type:

Standard Room - P2,500/day

Available Room

1 - Room 101

No. of Guests:

1

--Dates

Check-in Date

2026-05-17

Check-out Date

2026-05-17

Payment

Discount

☐ Senior / PWD

Payment Method

Cash

Total Amount

Standard Room



Clear Form

Save Booking

Room Availability

Sync Suites Hotel

Logout

New Booking

Room Availability

Manage Booking

Room Availability Overview

Room No.	Room Type	Rate / Day (P=)	Status
106	Standard Room	2500.00	Available
107	Standard Room	2500.00	Available
108	Standard Room	2500.00	Available
201	Junior Suite	3500.00	Available
202	Junior Suite	3500.00	Available
203	Junior Suite	3500.00	Available
204	Junior Suite	3500.00	Available
205	Junior Suite	3500.00	Available
206	Junior Suite	3500.00	Available
207	Junior Suite	3500.00	Available
208	Junior Suite	3500.00	Available
301	Executive Suite	5000.00	Available
302	Executive Suite	5000.00	Available
303	Executive Suite	5000.00	Available
304	Executive Suite	5000.00	Available
305	Executive Suite	5000.00	Available
101	Standard Room	2500.00	Reserved
102	Standard Room	2500.00	Reserved
103	Standard Room	2500.00	Reserved
104	Standard Room	2500.00	Reserved
105	Standard Room	2500.00	Reserved
401	Presidential Suite	8000.00	Reserved
403	Presidential Suite	8000.00	Reserved
404	Presidential Suite	8000.00	Reserved

Refresh

Manage Booking Panel

Sync Suites Hotel

Logout

New BookingRoom AvailabilityManage Booking

Manage Booking

Booking ID	Customer Name	Room No.	Room Type	Check-In	Check-Out	Total (P)	Payment	Status
37	HELLO WORLD	105	Standard Room	2026-05-18	2026-05-21	7200.0	Cash	Reserved
36	RENYER EXCLSV	104	Standard Room	2026-05-17	2026-05-19	5000.0	Cash	Reserved
31	RJ ESPIJON	404	Presidential Suite	2026-05-17	2026-05-18	9200.0	Cash	Reserved
33	SEAN UTR	101	Standard Room	2026-05-17	2026-05-18	2400.0	Cash	Reserved
28	JUNJUN BATUMBAKAL	401	Presidential Suite	2026-05-17	2026-05-18	6400.0	Cash	Reserved
22	RDA UYTT	101	Standard Room	2026-05-17	2026-05-18	2400.0	Cash	Checked Out
24	RYZEN SHUR	201	Junior Suite	2026-05-17	2026-05-18	3500.0	Cash	Checked Out
35	JANJAN VALZDEZ	103	Standard Room	2026-05-17	2026-05-18	3000.0	Cash	Reserved
30	SANTO NINO	403	Presidential Suite	2026-05-17	2026-05-18	7600.0	Cash	Reserved
25	RENYER EXCLSV	101	Standard Room	2026-05-17	2026-05-18	2500.0	Cash	Checked Out
27	RYZEN SHUR	401	Presidential Suite	2026-05-17	2044-11-17	5.4072E7	Cash	Checked Out
21	SEAN UTR	101	Standard Room	2026-05-17	2026-05-18	2500.0	Cash	Checked Out
32	KYZEN REYZ	405	Presidential Suite	2026-05-17	2026-05-18	6400.0	Cash	Reserved
34	JAYJO BATUMBAKAL	102	Standard Room	2026-05-17	2026-05-18	2500.0	Cash	Reserved
29	JOYJOY ODADTNAT	402	Presidential Suite	2026-05-17	2026-05-19	12800.0	GCashQR Scan	Checked In
23	RYZEN SHUR	101	Standard Room	2026-05-17	2026-05-18	2500.0	Cash	Checked Out
26	RYZEN SHUR	401	Presidential Suite	2026-05-17	2026-05-18	7500.0	Cash	Checked Out

Refresh

Check In

Check Out

Extend Stay

Upgrade Room

Reschedule

Cancel Booking

Filter